PYTHON ACTIVITY-BASED LEARNING

Student Name: G.lokesh

Roll No.:26001209

Selected Case Studies

1. University Scholarship Allocation
2. Progressive Electricity Billing
3. Hospital Billing and Patient Classification
4. Employee Payroll and Deduction Analysis
5. Multi-Plan Telecommunications Cost Optimization
6. Railway Fare and Reservation Calculator
7. Retail Billing with Multiple Discount Rules
8. Manufacturing Cost and Profitability Analysis
9. Multi-Currency Travel Budget Analysis
10. Campus Event Financial Planning

Note: The flowcharts are provided as readable text flow diagrams. The output screenshots are generated console-style illustrations of the sample outputs; actual console appearance may vary by Python environment.

1. University Scholarship Allocation

1. Problem Statement

Develop a program that checks scholarship applicants using aggregate marks, minimum subject marks, attendance, and family income. Eligible applicants are ranked by aggregate marks, then attendance.

2. Objectives

Check each applicant against all eligibility conditions.

Store applicant details using tuples inside a list.

Rank eligible applicants using clear tie-breaking rules.

3. Assumptions

Aggregate marks must be at least 75%.

Minimum subject mark must be at least 60%.

Attendance must be at least 75%.

Annual family income must not exceed ₹3,00,000.

Ties are resolved by higher attendance.

4. Algorithm

1. Start.

2. Store applicant records in a list of tuples.

3. Check marks, subject minimum, attendance, and income for each

applicant. 4. Add eligible applicants to a separate list.

5. Sort eligible applicants by aggregate and attendance, both in descending

order. 6. Display the ranked list; if empty, display no eligible applicants.

7. Stop.

5. Flowchart

START → Read applicant records → Check all four eligibility conditions → Eligible? → Yes: add to eligible list → Sort by aggregate, then attendance → Display ranking → END
No: skip applicant ↗

6. Pseudocode

```
BEGIN
eligible ← empty list
FOR each applicant in applicants
  IF aggregate ≥ 75 AND subject_min ≥ 60 AND attendance ≥ 75 AND income ≤ 300000
    append applicant to eligible
  ENDIF
ENDFOR
sort eligible by aggregate descending, attendance descending
display eligible
END
```

7. Python Source Code

```
applicants = [
    ("Anu", 88, 70, 90, 250000),
    ("Bala", 78, 62, 80, 280000),
    ("Charan", 91, 55, 95, 200000),
    ("Divya", 85, 68, 72, 180000)
]
```

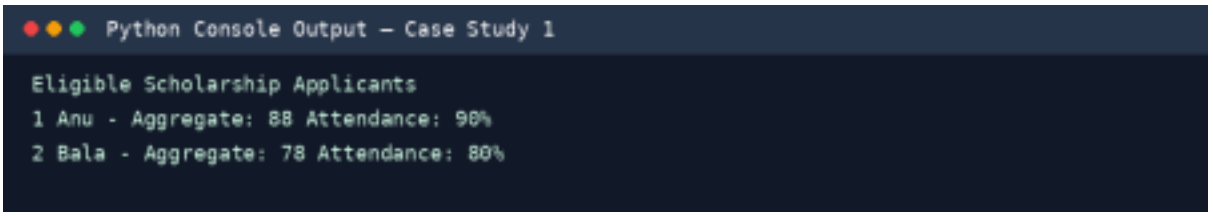
```
eligible = []
for name, aggregate, subject_min, attendance, income in applicants:
    if (aggregate >= 75 and subject_min >= 60
            and attendance >= 75 and income <= 300000):
        eligible.append((name, aggregate, attendance))
eligible.sort(key=lambda student: (student[1], student[2]), reverse=True)
print("Eligible Scholarship Applicants")
if eligible:
    for rank, student in enumerate(eligible, start=1):
        print(rank, student[0], "- Aggregate:", student[1],
              "Attendance:", str(student[2]) + "%")
else:
    print("No eligible applicants")
```

8. Test Cases

Test Input / Condition	Expected Result
Normal eligible applicant Anu: 88%, 70 minimum subject mark	Eligible; ranked first
Not eligible subject minimum, 90% attendance, ₹250000	
Subject score below 60 Charan has 55	Not eligible
Attendance below 75 Divya has 72% attendance	Not eligible

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Eligible Scholarship Applicants
1 Anu - Aggregate: 88 Attendance: 90%
2 Bala - Aggregate: 78 Attendance: 80%

10. Conclusion

The program applies every scholarship condition and produces a ranked list of eligible applicants using aggregate marks and attendance as tie-breakers.

2. Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using consumption slabs, a fixed charge, a surcharge for high bills, and a rebate for low consumption.

2. Objectives

Calculate energy charges slab by slab.

Apply fixed charges and the specified surcharge or rebate.

Display a clear bill breakdown.

3. Assumptions

First 100 units cost ₹1.50 per unit.

Units 101–200 cost ₹2.50 per unit.

Units above 200 cost ₹4.00 per unit.

Fixed charge is ₹50.

A 5% surcharge applies when the amount before surcharge exceeds ₹500; consumption up to 100 units receives a 10% rebate on energy charges instead.

4. Algorithm

1. Start and read units consumed.
2. Reject negative consumption.
3. Calculate energy charge using the three slabs.
4. Add ₹50 fixed charge.
5. Apply the surcharge or rebate rule.
6. Display units, energy charge, adjustment, and final bill.
7. Stop.

5. Flowchart

START → Input units → Units ≥ 0? → Calculate slab charges → Add fixed charge → Apply surcharge/rebate → Display final bill → END
No → Display invalid input → END

6. Pseudocode

```
BEGIN
READ units
IF units < 0: DISPLAY invalid input
ELSE
  calculate energy charge using slabs
  bill ← energy charge + 50
  IF bill > 500: bill ← bill × 1.05
  ELSE IF units ≤ 100: bill ← bill − 10% of energy charge
  DISPLAY bill
ENDIF
END
```

7. Python Source Code

```
units = int(input("Enter units consumed: "))
if units < 0:
    print("Invalid units")
else:
    if units <= 100:
        energy = units * 1.50
    elif units <= 200:
        energy = 100 * 1.50 + (units - 100) * 2.50
    else:
        energy = 100 * 1.50 + 100 * 2.50 + (units - 200) * 4.00
    bill = energy + 50
    adjustment = 0
    if bill > 500:
        adjustment = bill * 0.05
```

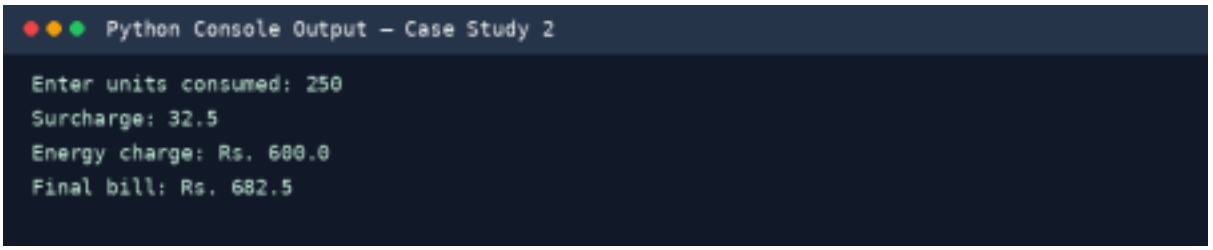
```
    bill += adjustment
    print("Surcharge:", round(adjustment, 2))
elif units <= 100:
    adjustment = energy * 0.10
    bill -= adjustment
    print("Rebate:", round(adjustment, 2))
print("Energy charge: Rs.", round(energy, 2))
print("Final bill: Rs.", round(bill, 2))
```

8. Test Cases

Test Input / Condition	Expected Result
Low use 80 units	Energy ₹120; rebate ₹12; final bill ₹158
High use 250 units	Energy ₹600; surcharge ₹32.50; final bill ₹682.50
Invalid input -5 units	Invalid units

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

```
Enter units consumed: 250
Surcharge: 32.5
Energy charge: Rs. 600.0
Final bill: Rs. 682.5
```

10. Conclusion

The program calculates bills fairly across consumption slabs and demonstrates conditional application of surcharges, rebates, and input validation.

3. Hospital Billing and Patient Classification

1. Problem Statement

Create a simple hospital billing program that adds consultation, diagnostic, and treatment charges, gives a concession to senior patients, and classifies the bill.

2. Objectives

Calculate total service charges.

Apply a concession when the age condition is met.

Classify the bill using a stated billing policy.

3. Assumptions

Consultation charge is ₹500, diagnostics ₹800, and treatment ₹1,500.

Patients aged 60 or above receive a 10% concession.

A final bill of ₹2,500 or more is classified as High Cost; otherwise it is Standard.

Negative age is invalid.

4. Algorithm

1. Start and read patient age.
2. Validate age.
3. Add the three service charges.
4. If age is at least 60, subtract a 10% concession.
5. Classify the final bill and display the bill details.
6. Stop.

5. Flowchart

START → Input age → Valid age? → Add service charges → Age ≥ 60? → Apply 10% concession if yes → Classify bill → Display → END

6. Pseudocode

```
BEGIN
READ age
IF age < 0: DISPLAY invalid age
ELSE
  total ← 500 + 800 + 1500
  IF age ≥ 60: concession ← total × 0.10
  ELSE: concession ← 0
  final_bill ← total − concession
  IF final_bill ≥ 2500: category ← High Cost
  ELSE: category ← Standard
  DISPLAY total, concession, final_bill, category
ENDIF
END
```

7. Python Source Code

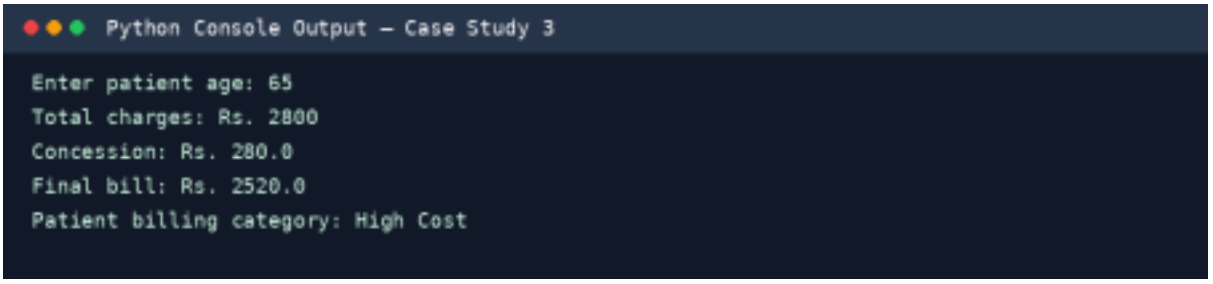
```
age = int(input("Enter patient age: "))
if age < 0:
    print("Invalid age")
else:
    consultation = 500
    diagnostics = 800
    treatment = 1500
    total = consultation + diagnostics + treatment
    concession = total * 0.10 if age >= 60 else 0
    final_bill = total - concession
    category = "High Cost" if final_bill >= 2500 else "Standard"
    print("Total charges: Rs.", total)
    print("Concession: Rs.", round(concession, 2))
    print("Final bill: Rs.", round(final_bill, 2))
    print("Patient billing category:", category)
```

8. Test Cases

Test Input / Condition	Expected Result
Senior patient Age 65	Total ₹2800; concession ₹280; final ₹2520; High Cost
Adult patient Age 35	Final ₹2800; High Cost
Invalid age Age -2	Invalid age

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Enter patient age: 65
Total charges: Rs. 2800
Concession: Rs. 280.0
Final bill: Rs. 2520.0
Patient billing category: High Cost

10. Conclusion

The program combines arithmetic and conditional statements to prepare a hospital bill, apply a senior-patient concession, and assign a billing category.

4. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate an employee's gross salary from basic pay, allowances, overtime, and incentives. Apply provident-fund and progressive tax deductions to calculate net salary.

2. Objectives

Compute allowances, overtime, and gross salary.

Apply provident-fund and progressive tax deductions.

Print a formatted salary statement.

3. Assumptions

HRA is 20% and DA is 10% of basic pay.

Overtime is paid at ₹150 per hour.

PF is 12% of basic pay.

Tax is 5% on the first ₹30,000 of gross salary and 10% on the portion above ₹30,000.

All inputs are non-negative.

4. Algorithm

1. Read basic pay, overtime hours, and incentive.

2. Validate that values are non-negative.

3. Calculate HRA, DA, overtime pay, and gross salary.

4. Calculate PF and progressive tax.

5. Subtract deductions from gross salary.

6. Display salary components and net salary.

7. Stop.

5. Flowchart

START → Read pay details → Inputs valid? → Calculate allowances and overtime → Gross salary → Calculate PF and tax → Net salary → Display statement → END

6. Pseudocode

```
BEGIN
READ basic, overtime_hours, incentive
IF any input < 0: DISPLAY invalid input
ELSE
  HRA ← basic × 0.20; DA ← basic × 0.10
  overtime_pay ← overtime_hours × 150
  gross ← basic + HRA + DA + overtime_pay + incentive
  PF ← basic × 0.12
  tax ← min(gross, 30000) × 0.05 + max(gross − 30000, 0) × 0.10
  net ← gross − PF − tax
  DISPLAY salary statement
ENDIF
END
```

7. Python Source Code

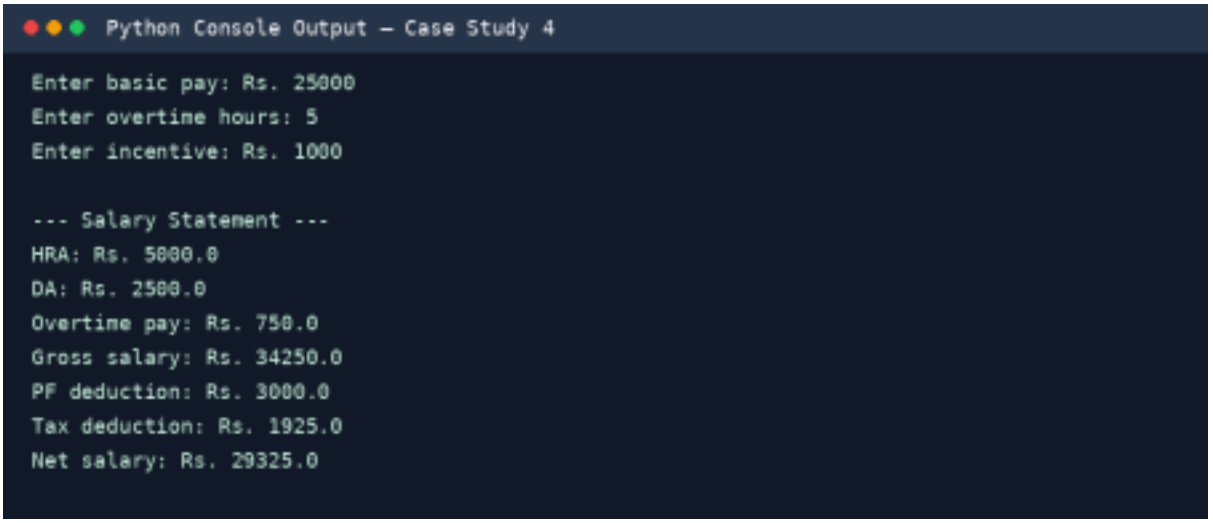
```
basic = float(input("Enter basic pay: Rs. "))
overtime_hours = float(input("Enter overtime hours: "))
incentive = float(input("Enter incentive: Rs. "))
if basic < 0 or overtime_hours < 0 or incentive < 0:
    print("Invalid input")
else:
    hra = basic * 0.20
    da = basic * 0.10
    overtime_pay = overtime_hours * 150
    gross = basic + hra + da + overtime_pay + incentive
    pf = basic * 0.12
    tax = min(gross, 30000) * 0.05 + max(gross - 30000, 0) * 0.10
    net = gross - pf - tax
    print("\n--- Salary Statement ---")
    print("HRA: Rs.", round(hra, 2))
    print("DA: Rs.", round(da, 2))
    print("Overtime pay: Rs.", round(overtime_pay, 2))
    print("Gross salary: Rs.", round(gross, 2))
    print("PF deduction: Rs.", round(pf, 2))
    print("Tax deduction: Rs.", round(tax, 2))
    print("Net salary: Rs.", round(net, 2))
```

8. Test Cases

Test Input / Condition	Expected Result
Standard payroll Basic ₹25000, overtime 5 hours, incentive ₹1000	Gross ₹34250; PF ₹3000; tax ₹1925; net ₹29325
No overtime Basic ₹20000, overtime 0, incentive ₹0	Overtime pay ₹0
Invalid input Basic pay -100	Invalid input

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Enter basic pay: Rs. 25000
Enter overtime hours: 5
Enter incentive: Rs. 1000
--- Salary Statement ---
HRA: Rs. 5000.0
DA: Rs. 2500.0
Overtime pay: Rs. 750.0
Gross salary: Rs. 34250.0
PF deduction: Rs. 3000.0
Tax deduction: Rs. 1925.0
Net salary: Rs. 29325.0

10. Conclusion

The program demonstrates how to calculate gross and net salary with allowances and progressive deductions, producing a useful payroll statement.

5. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare mobile plans for a customer's data usage, required validity, and budget. Include excess-data charges and select the lowest-cost plan that meets all constraints.

2. Objectives

Calculate total cost for each mobile plan.

Check validity and budget restrictions.
Choose the least expensive eligible plan.

3. Assumptions

A plan is eligible only if validity is at least the required days and total cost is within budget.

Excess data is charged for each GB above the included allowance.

Plan details are supplied as tuples: name, base price, included GB, excess price per GB, validity days.

4. Algorithm

1. Read data usage, required validity, and budget.

2. For each plan, calculate excess data and total cost.

3. Check validity and budget.

4. Store eligible plans with their total cost.

5. Select the eligible plan with the lowest total cost.

6. Display all eligible plans and the best plan; if none, report no

match. 7. Stop.

5. Flowchart

START → Input usage, validity, budget → Evaluate each plan → Calculate excess charge → Meets validity and budget? → Save eligible plan → Select minimum cost → Display result → END

6. Pseudocode

```
BEGIN
READ usage, required_days, budget
eligible ← empty list
FOR each plan
  excess ← max(usage − included_GB, 0)
  cost ← base_price + excess × excess_rate
  IF validity ≥ required_days AND cost ≤ budget: append plan and cost
ENDFOR
IF eligible is empty: DISPLAY no suitable plan
ELSE: select plan with minimum cost and DISPLAY it
END
```

7. Python Source Code

```
plans = [
   ("Plan A", 299, 10, 20, 28),
   ("Plan B", 399, 20, 0, 56),
   ("Plan C", 249, 5, 20, 28)
]
usage = float(input("Enter expected data usage (GB): "))
required_days = int(input("Enter required validity (days): "))
budget = float(input("Enter budget: Rs. "))
eligible = []
for name, base, included, extra_rate, validity in plans:
   extra_gb = max(usage - included, 0)
   total_cost = base + extra_gb * extra_rate
   if validity >= required_days and total_cost <= budget:
      eligible.append((name, total_cost, validity))
for plan in eligible:
   print(plan[0], "- Total cost: Rs.", round(plan[1], 2),
      "- Validity:", plan[2], "days")
if eligible:
   best = min(eligible, key=lambda item: item[1])
   print("Most economical plan:", best[0],
      "at Rs.", round(best[1], 2))
else:
   print("No plan satisfies the requirements")
```

8. Test Cases

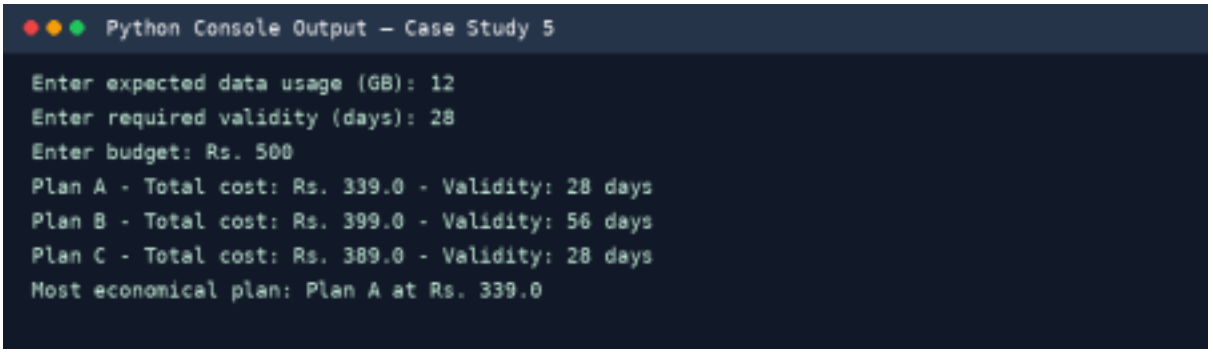
Test Input / Condition	Expected Result
Suitable plans	Usage 12 GB, 28 days, budget ₹500
Long validity	Usage 12 GB, 40 days, budget ₹500
Low budget	Usage 12 GB, 28 days, budget ₹200

9. Sample Output Screenshot

Plan A ₹339; Plan B ₹399; Plan C ₹389; choose Plan A Only Plan B qualifies on validity

No suitable plan

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Enter expected data usage (GB): 12
Enter required validity (days): 28
Enter budget: Rs. 500
Plan A - Total cost: Rs. 339.0 - Validity: 28 days
Plan B - Total cost: Rs. 399.0 - Validity: 56 days
Plan C - Total cost: Rs. 389.0 - Validity: 28 days
Most economical plan: Plan A at Rs. 339.0

10. Conclusion

The program compares plans consistently and chooses the lowest-cost option that satisfies the customer's usage, validity, and budget requirements.

6. Railway Fare and Reservation Calculator

1. Problem Statement

Calculate a passenger's railway fare based on travel distance, travel class, reservation fee, and an age-based concession.

2. Objectives

Calculate distance-based fares using slabs.

Apply a travel-class multiplier and senior concession.
Validate inputs and display the final fare.

3. Assumptions

First 100 km cost ₹1.20 per km; each additional km costs ₹1.00.

Class multipliers: Sleeper 1.0 and AC 1.8.

Reservation fee is ₹40 per passenger.

Passengers aged 60 or above receive a 20% concession on the distance fare

only. Distance must be positive and age non-negative.

4. Algorithm

1. Read distance, age, and class.

2. Validate the inputs and class.

3. Calculate the distance fare using slabs.

4. Multiply by the class factor.

5. Apply a 20% concession to the distance fare for age 60 or

above. 6. Add reservation fee and display final fare.

7. Stop.

5. Flowchart

START → Input distance, age, class → Validate → Calculate slab fare → Apply class multiplier → Senior? → Apply concession if yes → Add reservation fee → Display fare → END

6. Pseudocode

```
BEGIN
READ distance, age, travel_class
IF invalid input: DISPLAY error
ELSE
  IF distance ≤ 100: base ← distance × 1.20
  ELSE: base ← 120 + (distance − 100) × 1.00
  fare ← base × class_multiplier
  IF age ≥ 60: fare ← fare × 0.80
  final_fare ← fare + 40
  DISPLAY final_fare
ENDIF
END
```

7. Python Source Code

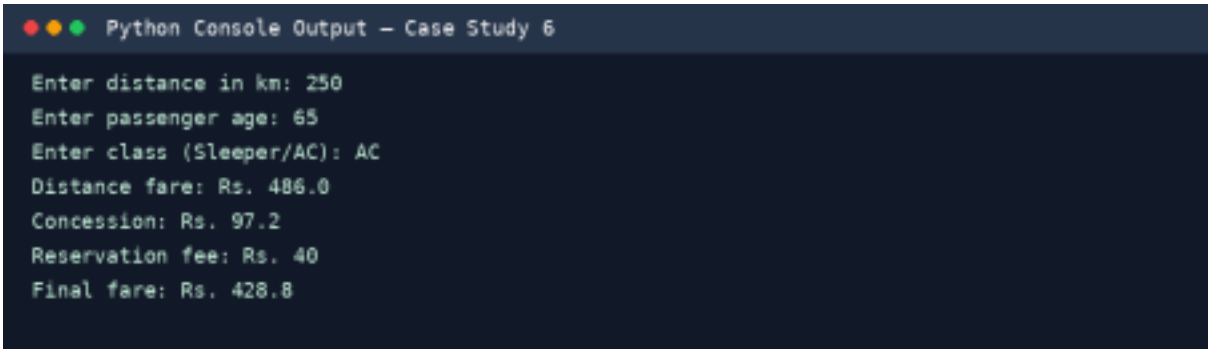
```
distance = float(input("Enter distance in km: "))
age = int(input("Enter passenger age: "))
travel_class = input("Enter class (Sleeper/AC): ").strip().lower()
if distance <= 0 or age < 0 or travel_class not in ("sleeper", "ac"):
    print("Invalid input")
else:
    if distance <= 100:
        base_fare = distance * 1.20
    else:
        base_fare = 100 * 1.20 + (distance - 100) * 1.00
    multiplier = 1.0 if travel_class == "sleeper" else 1.8
    distance_fare = base_fare * multiplier
    concession = distance_fare * 0.20 if age >= 60 else 0
    final_fare = distance_fare - concession + 40
    print("Distance fare: Rs.", round(distance_fare, 2))
    print("Concession: Rs.", round(concession, 2))
    print("Reservation fee: Rs. 40")
    print("Final fare: Rs.", round(final_fare, 2))
```

8. Test Cases

Test Input / Condition	Expected Result
Senior AC passenger 250 km, age 65, AC	Distance fare ₹486; concession ₹97.20; final ₹428.80
Sleeper passenger 80 km, age 25, Sleeper	Distance fare ₹96; final ₹136
Invalid class 250 km, age 25, First Class	Invalid input

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

```
Enter distance in km: 250
Enter passenger age: 65
Enter class (Sleeper/AC): AC
Distance fare: Rs. 486.0
Concession: Rs. 97.2
Reservation fee: Rs. 40
Final fare: Rs. 428.8
```

10. Conclusion

The program uses slab calculations, class multipliers, and age-based conditions to calculate a passenger's final railway fare.

7. Retail Billing with Multiple Discount Rules

1. Problem Statement

Prepare an itemized retail invoice for products in different tax categories. Apply a threshold discount subject to a maximum concession limit and calculate tax after the discount.

2. Objectives

Calculate item subtotals and total subtotal.

Apply the discount threshold and maximum limit.

Allocate discount proportionally and calculate category-specific tax.

3. Assumptions

Notebook: ₹100 each, 5% tax; Bag: ₹300 each, 12% tax.

A 10% discount applies when subtotal is at least ₹2,000.

The discount cannot exceed ₹500.

Discount is distributed proportionally across items before tax.

4. Algorithm

1. Store product, quantity, unit price, and tax rate.
2. Calculate each item's subtotal and overall subtotal.
3. Apply 10% discount if the threshold is met, capped at ₹500.
4. Allocate discount proportionally to each item.
5. Calculate tax on each discounted item and sum the invoice.
6. Display the invoice totals.
7. Stop.

5. Flowchart

START → Load item details → Calculate subtotals → Subtotal ≥ ₹2000? → Calculate capped discount → Allocate discount → Calculate item taxes → Print invoice → END

6. Pseudocode

```
BEGIN
items ← product, quantity, price, tax rate records
subtotal ← sum(quantity × price)
IF subtotal ≥ 2000: discount ← min(subtotal × 0.10, 500)
ELSE: discount ← 0
FOR each item
  share ← item_subtotal / subtotal
  discounted ← item_subtotal − discount × share
  tax ← discounted × tax_rate
  add discounted + tax to invoice total
ENDFOR
DISPLAY subtotal, discount, tax, invoice total
END
```

7. Python Source Code

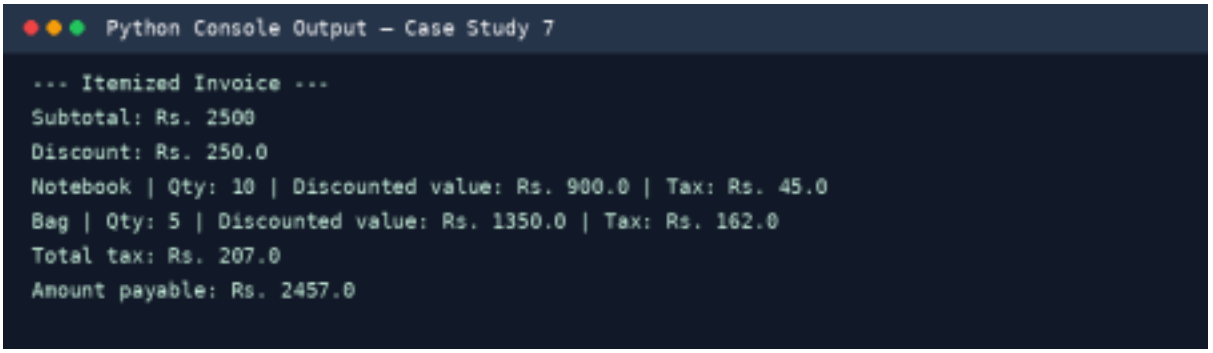
```
items = [
    ("Notebook", 10, 100, 0.05),
    ("Bag", 5, 300, 0.12)
]
subtotal = sum(qty * price for name, qty, price, rate in items)
discount = min(subtotal * 0.10, 500) if subtotal >= 2000 else 0
total_tax = 0
print("--- Itemized Invoice ---")
print("Subtotal: Rs.", round(subtotal, 2))
print("Discount: Rs.", round(discount, 2))
for name, qty, price, rate in items:
    item_subtotal = qty * price
    share = item_subtotal / subtotal if subtotal else 0
    discounted = item_subtotal - discount * share
    tax = discounted * rate
    total_tax += tax
    print(name, "| Qty:", qty, "| Discounted value: Rs.",
          round(discounted, 2), "| Tax: Rs.", round(tax, 2))
print("Total tax: Rs.", round(total_tax, 2))
print("Amount payable: Rs.", round(subtotal - discount + total_tax, 2))
```

8. Test Cases

Test Input / Condition	Expected Result
Discount applies 10 notebooks at ₹100 and 5 bags at ₹300	Subtotal ₹2500; discount ₹250; tax ₹207; payable ₹2457
Below threshold 1 notebook at ₹100 and 1 bag at ₹300	Subtotal ₹400; discount ₹0
Discount cap Subtotal ₹7000	Discount capped at ₹500

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

```
--- Itemized Invoice ---
Subtotal: Rs. 2500
Discount: Rs. 250.0
Notebook | Qty: 10 | Discounted value: Rs. 900.0 | Tax: Rs. 45.0
Bag | Qty: 5 | Discounted value: Rs. 1350.0 | Tax: Rs. 162.0
Total tax: Rs. 207.0
Amount payable: Rs. 2457.0
```

10. Conclusion

The program creates an itemized invoice and applies the discount limit correctly. Tax is calculated on each item's discounted value.

8. Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate manufacturing cost from materials, labour, machine costs, and material wastage. Compare the expected profit with a required profit target.

2. Objectives

Calculate material and wastage costs.

Find total production cost, sales revenue, and profit.
Determine whether the target profit is achieved.

3. Assumptions

Material cost equals units produced multiplied by material cost per unit.

Wastage is a percentage of material cost.

Total cost includes material, wastage, labour, and machine costs.

The target profit is 20% of total production cost.

4. Algorithm

1. Read production quantity, material cost per unit, labour, machine cost, wastage percentage, and selling price per unit.

2. Validate inputs.

3. Calculate material cost and wastage cost.

4. Calculate total production cost, revenue, and profit.

5. Calculate target profit and compare.

6. Display the result.

7. Stop.

5. Flowchart

START → Read production details → Validate → Calculate material and wastage cost → Calculate total cost and revenue → Calculate profit → Profit ≥ target? → Display result → END

6. Pseudocode

```
BEGIN
READ quantity, material_unit, labour, machine, wastage_percent, selling_unit
IF any input < 0: DISPLAY invalid input
ELSE
  material_cost ← quantity × material_unit
  wastage_cost ← material_cost × wastage_percent / 100
  total_cost ← material_cost + wastage_cost + labour + machine
  revenue ← quantity × selling_unit
  profit ← revenue − total_cost
  target ← total_cost × 0.20
  DISPLAY costs, revenue, profit
  IF profit ≥ target: DISPLAY target achieved
  ELSE: DISPLAY target not achieved
ENDIF
END
```

7. Python Source Code

```
quantity = int(input("Enter units produced: "))
material_unit = float(input("Material cost per unit: Rs. "))
labour = float(input("Total labour cost: Rs. "))
machine = float(input("Machine cost: Rs. "))
wastage_percent = float(input("Wastage percentage: "))
selling_unit = float(input("Selling price per unit: Rs. "))
values = [quantity, material_unit, labour, machine, wastage_percent, selling_unit]
if any(value < 0 for value in values):
    print("Invalid input")
else:
    material_cost = quantity * material_unit
    wastage_cost = material_cost * wastage_percent / 100
    total_cost = material_cost + wastage_cost + labour + machine
    revenue = quantity * selling_unit
```

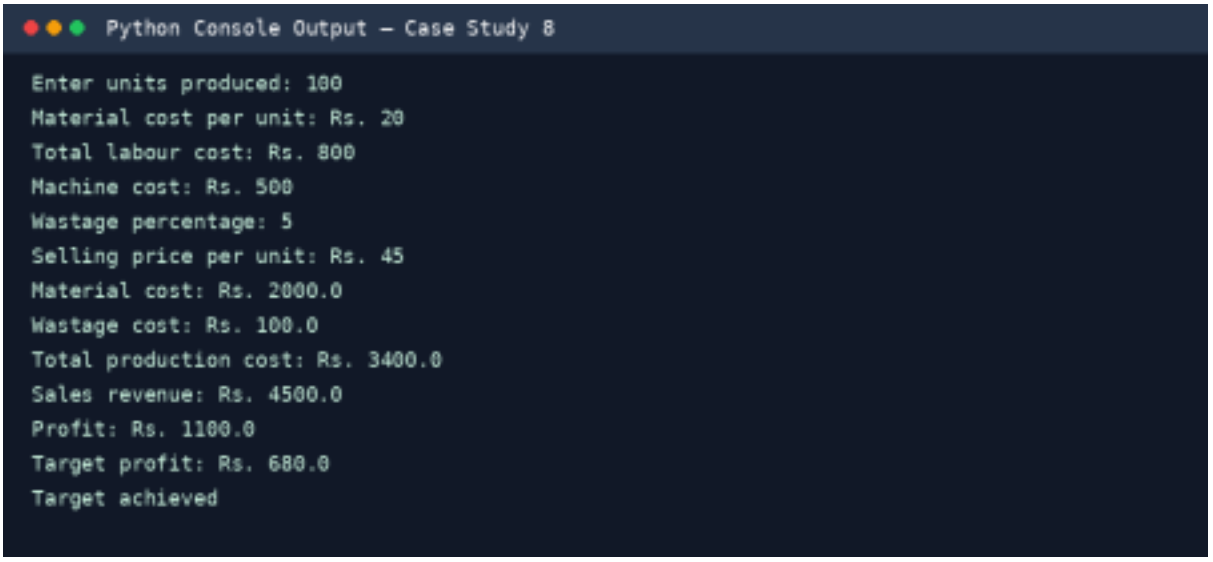
```
profit = revenue - total_cost
target = total_cost * 0.20
print("Material cost: Rs.", round(material_cost, 2))
print("Wastage cost: Rs.", round(wastage_cost, 2))
print("Total production cost: Rs.", round(total_cost, 2))
print("Sales revenue: Rs.", round(revenue, 2))
print("Profit: Rs.", round(profit, 2))
print("Target profit: Rs.", round(target, 2))
print("Target achieved" if profit >= target else "Target not achieved")
```

8. Test Cases

Test Input / Condition	Expected Result
Target achieved 100 units; material ₹20/unit; labour ₹800; machine ₹500; wastage 5%; selling ₹45/unit	₹50/unit Total cost ₹3400; revenue ₹4500; profit ₹1100; target achieved
Loss 10 units; material ₹100/unit; no other costs; selling	Profit is negative; target not achieved
Invalid input Negative quantity	Invalid input

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

```
Enter units produced: 100
Material cost per unit: Rs. 20
Total labour cost: Rs. 800
Machine cost: Rs. 500
Wastage percentage: 5
Selling price per unit: Rs. 45
Material cost: Rs. 2000.0
Wastage cost: Rs. 100.0
Total production cost: Rs. 3400.0
Sales revenue: Rs. 4500.0
Profit: Rs. 1100.0
Target profit: Rs. 680.0
Target achieved
```

10. Conclusion

The program measures production cost and profitability and clearly indicates whether the

required profit target has been reached.

9. Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses from a foreign currency into Indian rupees using a supplied exchange rate and conversion fee. Compare the final spending with a budget and identify the largest expense category.

2. Objectives

Convert expenses into INR.

Include a currency conversion fee.

Calculate total spending, budget balance, and largest category.

3. Assumptions

All listed expenses are in USD.

The exchange rate is ₹83 per USD.

A 2% conversion fee is applied to the converted amount.

The budget is ₹30,000.

4. Algorithm

1. Store travel expenses by category.

2. Convert each expense to INR using the exchange rate.

3. Calculate the conversion fee for each expense.

4. Add converted amounts and fees.

5. Find the category with the highest converted expense before

fee. 6. Compare total spending with budget and display results.

7. Stop.

5. Flowchart

START → Read/store expense categories → Convert USD to INR → Add 2% fee → Sum expenses → Find largest category → Compare with budget → Display report → END

6. Pseudocode

```
BEGIN
expenses ← category and USD amount records
FOR each expense
  converted ← amount × 83
  fee ← converted × 0.02
  total ← total + converted + fee
  store converted amount by category
ENDFOR
```

largest ← category with maximum converted amount
IF total ≤ budget: DISPLAY within budget
ELSE: DISPLAY over budget and excess
DISPLAY total and largest
END

7. Python Source Code

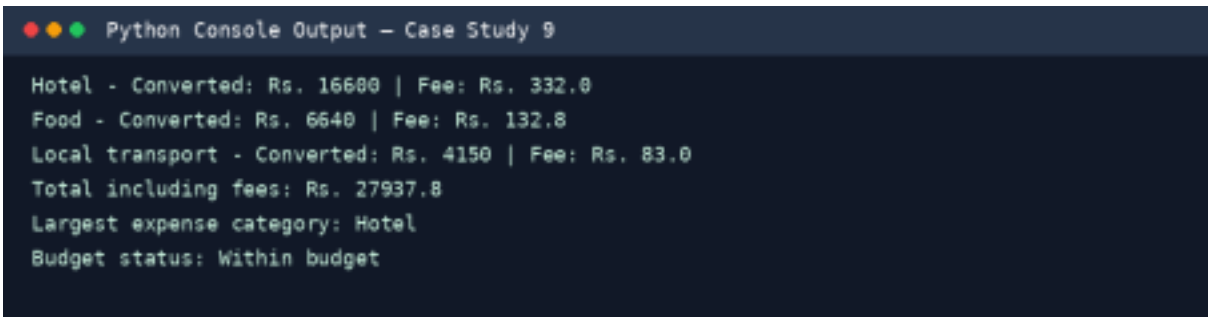
```
expenses = [
    ("Hotel", 200),
    ("Food", 80),
    ("Local transport", 50)
]
exchange_rate = 83
fee_rate = 0.02
budget = 30000
total = 0
converted_expenses = []
for category, usd in expenses:
    converted = usd * exchange_rate
    fee = converted * fee_rate
    amount_with_fee = converted + fee
    total += amount_with_fee
    converted_expenses.append((category, converted, fee))
    print(category, "- Converted: Rs.", round(converted, 2),
          "| Fee: Rs.", round(fee, 2))
largest = max(converted_expenses, key=lambda item: item[1])
print("Total including fees: Rs.", round(total, 2))
print("Largest expense category:", largest[0])
if total <= budget:
    print("Budget status: Within budget")
else:
    print("Budget status: Over budget by Rs.", round(total - budget, 2))
```

8. Test Cases

Test Input / Condition	Expected Result
Within budget Hotel $200, food $80, transport $50	Total including fees ₹27,937.80; largest Hotel; within ₹30,000
Over budget Hotel $300, food $100, transport $100	Over budget; excess displayed
Zero expense All categories $0	Total ₹0

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Hotel - Converted: Rs. 16600 | Fee: Rs. 332.0
Food - Converted: Rs. 6640 | Fee: Rs. 132.8
Local transport - Converted: Rs. 4150 | Fee: Rs. 83.0
Total including fees: Rs. 27937.8

Largest expense category: Hotel
Budget status: Within budget

10. Conclusion

The program converts travel expenses, includes conversion fees, identifies the largest spending category, and checks whether the trip stays within budget.

10. Campus Event Financial Planning

1. Problem Statement

Estimate the finances for a campus event by calculating fixed and per-participant costs, registration income, sponsorship, financial balance, and the minimum registrations required to break even.

2. Objectives

Calculate total event cost and income.

Determine surplus or shortfall.

Calculate the minimum number of registrations needed to break even.

3. Assumptions

Fixed costs: venue ₹12,000, equipment ₹5,000, and transport ₹3,000.

Variable cost per participant: catering ₹150 and certificate ₹20.

Registration fee is ₹500 per participant.

Sponsorship is ₹10,000.

Break-even assumes each additional registration contributes ₹500 minus ₹170 in variable cost.

4. Algorithm

1. Read the expected number of participants.

2. Validate that the number is non-negative.

3. Calculate fixed costs and variable costs.

4. Calculate registration income plus sponsorship.

5. Find the surplus or shortfall.

6. Calculate break-even registrations using fixed cost minus sponsorship divided by contribution per participant, rounded up.

7. Display the report.

8. Stop.

5. Flowchart

START → Input participants → Valid? → Calculate fixed and variable costs → Calculate income → Balance = income − costs → Calculate break-even count → Display report → END

6. Pseudocode

```
BEGIN
READ participants
IF participants < 0: DISPLAY invalid input
ELSE
  fixed_cost ← 12000 + 5000 + 3000
  variable_per_person ← 150 + 20
  total_cost ← fixed_cost + participants × variable_per_person
  income ← 10000 + participants × 500
  balance ← income − total_cost
  break_even ← ceiling((fixed_cost − 10000) / (500 − variable_per_person))
  DISPLAY total_cost, income, balance, break_even
ENDIF
END
```

7. Python Source Code

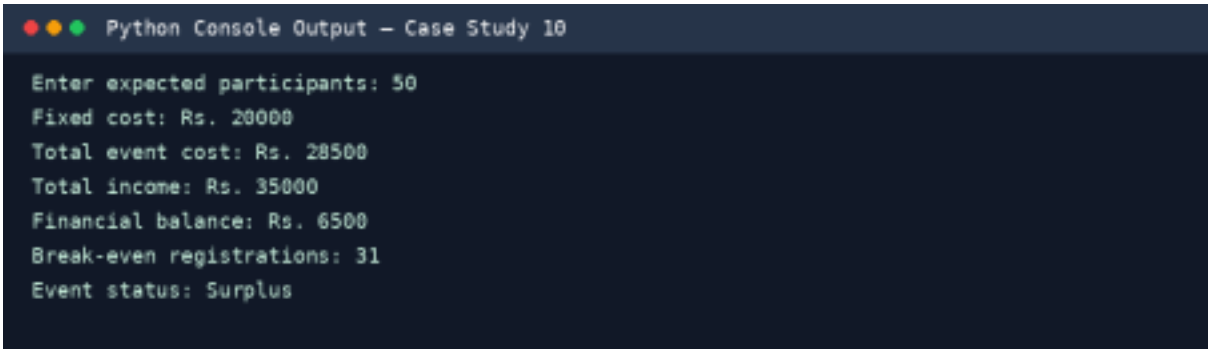
```
participants = int(input("Enter expected participants: "))
if participants < 0:
    print("Invalid participant count")
else:
    venue = 12000
    equipment = 5000
    transport = 3000
    catering_per_person = 150
    certificate_per_person = 20
    registration_fee = 500
    sponsorship = 10000
    fixed_cost = venue + equipment + transport
    variable_per_person = catering_per_person + certificate_per_person
    total_cost = fixed_cost + participants * variable_per_person
    income = sponsorship + participants * registration_fee
    balance = income - total_cost
    contribution = registration_fee - variable_per_person
    break_even = math.ceil((fixed_cost - sponsorship) / contribution)
    print("Fixed cost: Rs.", fixed_cost)
    print("Total event cost: Rs.", total_cost)
    print("Total income: Rs.", income)
    print("Financial balance: Rs.", balance)
    print("Break-even registrations:", break_even)
    if balance >= 0:
        print("Event status: Surplus")
    else:
        print("Event status: Shortfall")
```

8. Test Cases

Test Input / Condition	Expected Result
50 participants	Participants = 50 Cost ₹28,500; income ₹35,000; surplus ₹6,500; break-even 31
Break-even point	Participants = 31 Cost ₹25,270; income ₹25,500; surplus ₹230
Invalid input	Participants = -1 Invalid participant count

9. Sample Output Screenshot

The following image shows the expected sample console output for the example inputs.



Sample Output (Text)

Enter expected participants: 50
Fixed cost: Rs. 20000
Total event cost: Rs. 28500
Total income: Rs. 35000
Financial balance: Rs. 6500
Break-even registrations: 31
Event status: Surplus

10. Conclusion

The program supports campus event planning by estimating costs and income, identifying surplus or shortfall, and calculating the minimum registration count required for break-even.

Submission Checklist

- Run each Python program in IDLE, VS Code, or another Python 3 environment.
- Check the actual output against the test cases.
- Replace the sample console illustrations with your own screenshots if your faculty requires screenshots captured directly from your computer.
- Add your name, roll number, branch, and semester.
- Upload the final materials to the GitHub repository named Python-Activity-Based-Learning, as stated in the supplied assignment.